Observability: metrics, logs, traces và alert

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Logs, metrics và traces trả lời câu hỏi khác nhau

Log là bản ghi một sự kiện: request thất bại, exception, phiên bản khởi động, kết nối database lỗi. Log nên có thời gian, mức độ, tên dịch vụ và mã tương quan request khi phù hợp. Không ghi password/token hay dữ liệu cá nhân không cần thiết. Metrics là số đo theo thời gian: số request, tỷ lệ lỗi, độ trễ, CPU, memory, queue depth. Traces cho thấy một request đi qua nhiều dịch vụ và mất thời gian ở từng đoạn. Không cần cài toàn bộ công cụ ngay; hãy bắt đầu bằng câu hỏi điều tra cụ thể.

Một dashboard tốt giúp trả lời: người dùng có truy cập được không, lỗi tăng từ lúc nào, phần nào thay đổi, hệ thống có đủ tài nguyên không? Tránh dashboard chỉ hiển thị CPU vì CPU thấp không chứng minh API hoạt động. Đo cả tín hiệu người dùng như tỷ lệ request thành công và độ trễ. Phân biệt triệu chứng với nguyên nhân: tỷ lệ 5xx tăng là triệu chứng; nguyên nhân có thể là database pool cạn, cấu hình sai hoặc dependency chậm.

Cardinality là số chuỗi metrics khác nhau do các nhãn tạo ra. Gắn user ID hoặc request ID vào metric có thể tạo quá nhiều chuỗi và tốn tài nguyên. Đặt thông tin chi tiết này trong log/trace, còn metric dùng nhãn có tập giá trị giới hạn như service, endpoint nhóm hoặc status class.

Lab quan sát

Điều kiện: ứng dụng mẫu có log và endpoint. Tạo 10 request thành công và 2 request lỗi có kiểm soát. Ghi số lượng, status, thời điểm và thông báo log. Tính tỷ lệ thành công trong tập lab, nhưng không gọi con số đó là SLO sản xuất. Thêm một request ID hoặc correlation ID nếu ứng dụng hỗ trợ; lần theo cùng ID qua log. Sau đó cố ý làm database không sẵn sàng trong môi trường thử, quan sát tín hiệu nào xuất hiện trước và tín hiệu nào thiếu.

SLI, SLO và alert đáng hành động

SLI là chỉ số thể hiện chất lượng dịch vụ theo góc nhìn người dùng, ví dụ tỷ lệ request hợp lệ được trả thành công hoặc độ trễ dưới ngưỡng. SLO là mục tiêu cho SLI trong một khoảng thời gian. Error budget là phần sai lệch được phép khi so với mục tiêu. Ví dụ minh họa: nếu SLO là 99,9% request hợp lệ thành công trong 30 ngày, budget lý thuyết là 0,1% request không đạt; cách tính thực tế phải định nghĩa rõ request nào thuộc mẫu và cách xử lý lưu lượng thấp.

Không phải metric nào cũng nên gọi người trực. Alert tốt chỉ tới vấn đề có tác động đáng kể và có hành động khả thi. Nếu mỗi lần CPU tăng là một cuộc gọi nhưng dịch vụ vẫn ổn, nhóm sẽ mệt và bỏ qua cảnh báo quan trọng. Với hệ thống thực, cảnh báo có thể dựa trên tốc độ đốt error budget; cần thử ngưỡng và giảm nhiễu theo dữ liệu thật.

Bài tập định nghĩa SLO

Cho API ghi chú, hãy viết: ai là người dùng, request nào hợp lệ, "thành công" là status nào, thời gian đo là bao lâu và dữ liệu lấy từ đâu. Sau đó nêu hai giới hạn của chỉ số: có thể bỏ sót trải nghiệm nào? Nếu database không có request trong 10 phút, liệu 100% thành công có chứng minh dịch vụ tốt không? Đây là bài tập tư duy trước khi chọn Prometheus hoặc công cụ khác.

Tính SLI và error budget bằng ví dụ cụ thể

Giả sử trong 30 ngày API nhận 100.000 request hợp lệ. Có 99.850 request được trả thành công theo định nghĩa nhóm đã thống nhất. Availability SLI = 99.850 / 100.000 = 99,85%. Nếu SLO là 99,9%, error budget cho 100.000 request là 100 request không đạt; thực tế có 150 request không đạt, tức đã vượt budget 50. Trong dịch vụ thật, số request thay đổi liên tục nên hệ thống tính theo cửa sổ thời gian và quy tắc lọc rõ ràng. Ví dụ này chỉ giúp hiểu phép tính.

Định nghĩa "thành công" cần thận trọng. Một HTTP 200 có thể trả dữ liệu sai; một HTTP 404 do người dùng yêu cầu ID không tồn tại có thể là hành vi đúng, không nên tính là lỗi hệ thống. Nếu client hủy request, có tính vào mẫu không? Nếu chỉ đo từ server, lỗi DNS trước khi tới server sẽ không xuất hiện. Hãy ghi phạm vi chỉ số và giới hạn của nguồn dữ liệu.

Latency SLI có thể là tỷ lệ request dưới một ngưỡng thay vì chỉ lấy trung bình. Ví dụ 99 request nhanh và 1 request rất chậm có thể tạo trung bình trông ổn nhưng vẫn có người dùng trải nghiệm kém. Tập nhìn phân bố, percentile và số lượng mẫu; percentile trên ít request có thể dao động mạnh. Đừng đặt mục tiêu bằng con số tùy hứng: cần nhu cầu người dùng và dữ liệu thực tế.

Bài tập tính và diễn giải

Trong 10.000 request, có 9.990 request được xem là thành công. Nếu SLO là 99,95%, budget là 5 lỗi; thực tế có 10 lỗi, vượt budget 5. Hãy tự tính trước khi đọc câu giải. Nếu cả 10 lỗi xảy ra trong một phút, trải nghiệm người dùng khác với 10 lỗi rải đều; vì vậy alert còn cần xem tốc độ lỗi, không chỉ tổng tháng.

Từ log thô đến một cuộc điều tra có giả thuyết

Giả sử 09:15 có release mới. Từ 09:17, 5xx tăng từ gần 0 lên 8%, trong khi GET /health vẫn trả 200. Log cho thấy **sqlite3.OperationalError: unable to open database file** ở request POST /notes. Dữ kiện này gợi ý đường ghi dữ liệu lỗi, không phải toàn bộ HTTP server chết. Các giả thuyết hợp lý: thư mục DB không tồn tại, volume chưa mount, quyền thư mục sai hoặc biến **NOTES_DB** đổi. Bạn có thể kiểm tra cấu hình và quyền trong môi trường test, đối chiếu thay đổi release; nếu ảnh hưởng người dùng thật và rollback an toàn, phục hồi trước rồi tìm nguyên nhân chi tiết sau.

Một log tốt có thời gian, service, version, request ID và lỗi đủ rõ. Nhưng log thô có thể quá nhiều. Metric **http_requests_total{status_class="5xx"}** cho xu hướng; trace của request cho biết đoạn nào chậm hoặc lỗi. Mỗi công cụ trả lời một câu hỏi. Tập viết giả thuyết trước khi mở dashboard: "Nếu volume mount sai, log POST sẽ báo lỗi file và GET /health vẫn hoạt động". Sau đó tìm bằng chứng ủng hộ hoặc bác bỏ.

Bài tập diễn giải dashboard

Bạn thấy latency p95 tăng nhưng 5xx vẫn thấp. Những giả thuyết đầu tiên có thể là database chậm, hàng đợi dài hoặc dependency ngoài chậm. CPU cao có thể liên quan nhưng không đủ để kết luận. Hãy chọn ba phép đo tiếp theo và nêu vì sao: thời gian truy vấn DB, số kết nối chờ, độ trễ theo endpoint hoặc trace của request chậm. Nếu latency chỉ tăng ở POST, phạm vi điều tra thu hẹp hơn toàn ứng dụng.

Phòng ngừa alert mệt mỏi và vận hành có trách nhiệm

Một alert nên có người nhận, mức ưu tiên, điều kiện, hành động đầu tiên và cách tắt sau khi vấn đề đã được xử lý. Nếu alert đêm nào cũng nổ vì CPU tăng ngắn nhưng không có tác động, nó làm nhóm mất niềm tin. Trước khi nâng ngưỡng bừa bãi, xem service có thực sự bị ảnh hưởng không, metric có nhiễu không, khoảng đo có phù hợp không và alert có nên chuyển thành dashboard/ticket thay vì gọi người trực.

Trong postmortem, hành động cải thiện phải kiểm tra được. "Cẩn thận hơn" không phải hành động kỹ thuật rõ. "Thêm test kiểm tra quyền ghi volume trong staging trước release", "ghi version vào log khởi động" hoặc "diễn tập restore mỗi quý" là những việc có owner và bằng chứng. Ưu tiên hành động giảm khả năng lặp hoặc giảm thời gian phát hiện/khôi phục, cân đối với chi phí.

Mini review cuối giai đoạn

Mở ứng dụng mẫu và tự trả lời bằng bằng chứng: bản nào đang chạy; request nào lỗi; log/metric nào cho thấy; nếu database file mất thì restore thế nào; nếu bản deploy mới lỗi thì rollback ra sao; nếu cảnh báo nổ nhưng người dùng vẫn ổn thì bạn làm gì. Nếu chỉ trả lời bằng lý thuyết, làm thêm một diễn tập local rồi ghi kết quả.

Metric, log, trace: ba góc nhìn khác nhau

Metric là số liệu tổng hợp theo thời gian: request rate, error rate, latency percentile, CPU, queue depth. Log là bản ghi sự kiện có ngữ cảnh: request ID, phiên bản, lỗi cụ thể. Trace liên kết các span của cùng một request qua nhiều dịch vụ. Metric cho biết **có vấn đề**; trace gợi ý **đoạn nào chậm**; log giải thích **sự kiện cụ thể**. Không có một loại tín hiệu nào thay hoàn toàn hai loại kia.

Prometheus thường scrape endpoint metrics và lưu time series với labels. Grafana truy vấn và hiển thị, nhưng dashboard không tự tạo metric. Label có cardinality quá cao, ví dụ gắn user ID hoặc request ID vào mỗi time series, làm tăng số series và chi phí. Dùng request ID trong log/trace; dùng route template như **/notes/{id}** làm metric label thay vì path chứa mọi ID. Alert nên gắn với triệu chứng người dùng, như tỷ lệ lỗi và độ trễ so với SLO, rồi thêm alert hạ tầng có hành động rõ. Alert cho mọi CPU spike dễ gây nhiễu.

ELK/ECK/EFK gom log: shipper/collector gửi, Elasticsearch lưu và index, Kibana truy vấn; ECK là operator quản lý Elastic trên Kubernetes. Loki chọn mô hình log khác: labels cho stream và nội dung log được tìm kiếm theo cách phù hợp; vẫn phải hạn chế cardinality của labels. Zabbix thường mạnh cho giám sát hạ tầng truyền thống, item/trigger/host. Không nên nói "có Prometheus thì không cần Zabbix" nếu còn network device/VM và team đã vận hành Zabbix tốt. Chọn theo câu hỏi cần trả lời, chi phí lưu giữ, quyền truy cập log và khả năng vận hành.

OpenTelemetry cung cấp API/SDK và collector cho telemetry, giúp thu thập/biến đổi/xuất metric, log, trace tới backend. OTel không phải dashboard hay hệ lưu trữ cuối cùng. Cần quyết định sampling cho trace, correlation giữa request ID/trace ID, retention và loại bỏ dữ liệu cá nhân hoặc secret khỏi telemetry.

Lab 2 - Điều tra một request lỗi bằng ba loại tín hiệu

Giả sử dashboard cho thấy p95 latency tăng từ 200 ms lên 2 s sau release. Trace của request chậm có span DB 1.7 s; log ghi query timeout và version **v2**; metric DB cho thấy connection pool chạm trần. Viết giả thuyết, bằng chứng hỗ trợ và kiểm tra tiếp. Có thể lỗi do query mới, connection leak hoặc DB chậm; trace và metric chưa đủ để chọn một. Đề xuất mitigations như rollback v2 hoặc giới hạn traffic, rồi cách xác nhận p95/error rate hồi phục. Đừng chỉ nhìn CPU pod.

Azure Monitor, App Insights và KQL

Trong nhánh Azure, Application Insights quan sát ứng dụng qua request/dependency/exception; Log Analytics Workspace lưu log có thể truy vấn bằng Kusto Query Language (KQL); Azure Monitor Alerts chạy rule trên metric hoặc query. Quan hệ giữa resource, workspace và bảng log tùy cấu hình, nên khi điều tra cần xác nhận app đang gửi telemetry vào workspace nào và có sampling/retention ra sao. Một biểu đồ trống có thể do không thu thập, sai workspace, sai time range, quyền truy cập hoặc query sai; đừng kết luận "không có lỗi".

Ví dụ KQL mang tính minh họa, cần đối chiếu tên bảng/schema của workspace đang dùng:

```
AppRequests
| where TimeGenerated > ago(1h)
| summarize total=count(), failed=countif(Success == false) by bin(TimeGenerated, 5m)
| extend error_rate = todouble(failed) / total
```

Để điều tra một sự cố, lọc time range, service/version, operation/trace ID, rồi xem dependency và exception liên quan. KQL là công cụ đặt câu hỏi; chọn sai mẫu dữ liệu hoặc không tính sampling sẽ làm tỷ lệ sai. Với alert, ghi rõ ngưỡng, thời gian đánh giá, người nhận, runbook và cách kiểm tra alert đã hoạt động.

Case study có lời giải: p95 tăng nhưng CPU bình thường

Bối cảnh: p95 latency tăng từ 180 ms lên 1,8 s, CPU pod 35%, error rate chưa tăng. Đừng loại trừ sự cố vì CPU thấp. Xem latency theo route, theo version và theo upstream; trace cho thấy 1,5 s chờ SQL; DB metric báo connection pool đang chờ. Từ đây cần phân biệt query chậm, số connection bị giữ quá lâu hoặc pool quá nhỏ so với tải. Xem slow query/lock, active vs idle connection, tốc độ request và thay đổi gần thời điểm tăng. Nếu có query mới, rollback hoặc tối ưu query có thể giảm tác động nhanh; chỉ tăng pool có thể làm DB quá tải.

Alert tốt hơn: nếu SLO latency bị đốt nhanh trong một khoảng xác định và đủ traffic, báo người vận hành cùng link dashboard, query/trace mẫu và runbook. Đừng alert mỗi lần một request đơn lẻ chậm; cũng đừng chờ đến 5xx vì trải nghiệm đã xấu trước đó. Sau khôi phục, so p95, throughput, pool wait và phản hồi người dùng với baseline.

Thiết kế từ câu hỏi vận hành, không từ dashboard đẹp

Hỏi trước: khi người dùng báo “tạo note chậm”, bạn cần biết request nào, từ lúc nào, bao nhiêu người bị ảnh hưởng và nó chậm ở API, DB hay mạng? **Metric** trả lời xu hướng/tỷ lệ trên nhiều request; **log** giữ sự kiện chi tiết; **trace** nối một request qua nhiều bước. Không loại nào thay được loại còn lại. CPU thấp không chứng minh app nhanh; log đầy đủ nhưng thiếu request ID vẫn khó ghép chuỗi; trace mẫu có thể không bắt đúng request lỗi nếu sampling thấp.

Định nghĩa một SLI cụ thể cho **POST /notes**: tỷ lệ request hợp lệ trả 2xx/3xx trong dưới 500 ms tại edge, trong cửa sổ 30 ngày. Phải quyết định request 4xx do client có đưa vào mẫu số không, timeout đo ở đâu, maintenance có loại trừ không. SLO 99,9% trên 1.000.000 request đủ điều kiện cho tối đa khoảng 1.000 request “không tốt” theo định nghĩa này. Nếu 500 request hỏng trong một giờ, bạn đã tiêu nửa error budget tháng dù availability tính theo phút có thể kể khác. Một con số không có định nghĩa là khó dùng để quyết định release.

Cardinality là chi phí và rủi ro

Metric **http_requests_total{route="/notes",status="500"}** có số tổ hợp label vừa phải. Nếu thêm **user_id** hoặc raw URL chứa ID, số series có thể tăng theo số người dùng/request, gây tốn bộ nhớ/chi phí và query chậm. Dùng route template (**/notes/{id}**), không dùng path thực làm label. Log có thể chứa ID chi tiết với chính sách bảo vệ dữ liệu và retention phù hợp; trace ID là cầu nối từ metric spike sang một request mẫu. Thiết kế telemetry cùng code/pipeline, không đợi sự cố mới thêm.

Lab phân tích một sự cố giả định bằng ba tín hiệu

Dữ kiện: trước release, 10.000 req/5 phút, 20 lỗi 5xx, p95 180 ms. Sau release, 10.000 req/5 phút, 600 lỗi 5xx, p95 1,8 s, CPU API 35%, DB connection pool gần đầy. Tỷ lệ lỗi tăng từ 0,2% lên 6%. Nếu SLO là 99,9% request tốt, cửa sổ 5 phút này tiêu 600 lỗi trên 10.000 request; cần xem định nghĩa lỗi thực của SLI. Bạn không thể nói “CPU thấp nên không sao”.

Bước 1: dashboard metric xác định bắt đầu tăng từ revision nào và route nào. **Bước 2:** lọc log theo route, status và request ID; tìm lỗi **connection acquisition timeout** nếu có, không tự bịa log khi chưa quan sát. **Bước 3:** trace một request chậm; nếu phần lớn thời gian chờ lấy DB connection, giả thuyết mạnh là pool/leak/query chậm. **Bước 4:** đọc DB pool active/idle, DB connection count, slow query và thay đổi release. **Bước 5:** giảm tác động theo runbook, sau đó xác nhận error rate/p95 phục hồi. Viết rõ bằng chứng nào bác bỏ giả thuyết pool.

Prometheus, Grafana, logs và OpenTelemetry đặt ở đâu?

Prometheus scrape metric từ target hoặc exporter; Grafana truy vấn/hiển thị và có thể hỗ trợ alert tùy cấu hình. Alertmanager nhận alert từ Prometheus, gom nhóm, route, silence/inhibit theo chính sách. Loki/ELK/ECK lưu/truy vấn log theo mô hình khác; OpenTelemetry chuẩn hóa thu thập traces/metrics/logs qua SDK/Collector và backend. Không coi “đã cài Prometheus/Grafana” là có observability: cần metric ứng dụng, ownership, retention, dashboard đúng câu hỏi và alert có người xử lý.

Khi triển khai ECK trên Kubernetes, Elasticsearch cần storage bền, capacity RAM/disk, chính sách index/retention, backup snapshot và kiểm soát quyền. Log pipeline có thể mất dữ liệu khi buffer đầy hoặc network đứt; phải biết yêu cầu mất mát chấp nhận được và alert cho chính pipeline. Một cluster logging production là workload stateful riêng, không phải vài YAML có thể thêm vào cuối lab K8s.

Alert đáng đánh thức người trực ca

Một page nên gắn với triệu chứng người dùng hoặc rủi ro sắp gây ảnh hưởng, có ngưỡng, cửa sổ, severity, owner và hành động đầu tiên. “CPU > 80% trong 1 phút” thường không nói người dùng có đau không và có thể gây alert fatigue. Error rate/latency và burn rate theo SLO cho quyết định rõ hơn, cộng thêm capacity alert trước khi hết disk. Prometheus khuyến nghị ít alert, ưu tiên triệu chứng và có dashboard/runbook hữu ích; xem alerting practices. Cần metamonitoring: nếu hệ thống giám sát chết, ai phát hiện?

Bài tập: viết một alert cho **POST /notes** lỗi cao trong 10 phút. Ghi numerator/denominator, số request tối thiểu để tránh nhiễu khi traffic thấp, link dashboard, runbook, owner và điều kiện tắt page sau khi phục hồi. Đọc thử với tình huống chỉ có 2 request và 1 lỗi: tỷ lệ 50% không nhất thiết đáng gọi người dậy. Đọc Prometheus alerting overview và OpenTelemetry docs.

Histogram và p95 phải đọc có ngữ cảnh

Nếu 95% request dưới 200 ms và 5% request mất 10 giây, trung bình có thể trông vừa phải nhưng p95/p99 cho thấy đuôi chậm. Ngược lại, p95 của một route có 10 request trong 5 phút rất nhiễu. Khi xem latency, tách route/method/status và traffic volume, tránh label user ID. Histogram bucket phải đủ độ phân giải quanh SLO threshold; bucket 0,1s và 10s không giúp đánh giá mục tiêu 500 ms chính xác. Nếu hệ thống dùng trace sampling, 1 trace chậm không đại diện thống kê toàn bộ; dùng metric để định lượng, trace để giải thích một mẫu.

Mẫu runbook alert

Alert **NotesApiHighErrorRate** nổ khi SLI 5xx/eligible requests vượt ngưỡng trong cửa sổ đủ dài và lưu lượng tối thiểu. Runbook mở dashboard theo route/revision, kiểm tra synthetic request, so deploy gần nhất, log request ID, DB/queue health, rồi chọn giảm tác động. Annotation chứa owner, link dashboard và cách silence hợp lệ khi bảo trì. Sau incident, xem alert có nổ sớm quá/muộn quá, có page trùng nhau không, và người trực ca có hành động rõ không. Một alert chỉ nói “CPU 80%” thường thiếu hành động.

Câu hỏi: log đầy đủ có thay metric không? Không; tổng hợp log khi incident có thể chậm/tốn và thiếu dữ liệu nếu pipeline lỗi. **Metric xanh có chứng minh người dùng ở ngoài vào được?** Không nếu chỉ scrape nội bộ; thêm blackbox/synthetic từ đường người dùng. Nguồn: Prometheus alerting practices.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.

- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://prometheus.io/docs/
- https://opentelemetry.io/docs/
- https://grafana.com/docs/grafana/latest/
- https://www.elastic.co/guide/en/cloud-on-k8s/current/index.html